به نام خدا

طراحی کامپیوتری سیستم‌های دیجیتال

پاییز ۱۴۰۵

پروژه دوم - فاز اول: سنتز دستی شتابدهنده عصبی با ماژول‌های Actel

طراحان : سپهر جمالی - محمد سپهر بازرگان - هاتف رضائی

هدف پروژه :

در این تمرین و تمرین بعدی آن (دو فاز مرتبط) می‌خواهیم با نحوه سنتز شدن کد وریلاگ روی FPGA آشنا شویم، به این منظور ماژول‌های تمرین یک را به صورت دستی توسط ماژول های Actel پیاده سازی می‌کنیم.

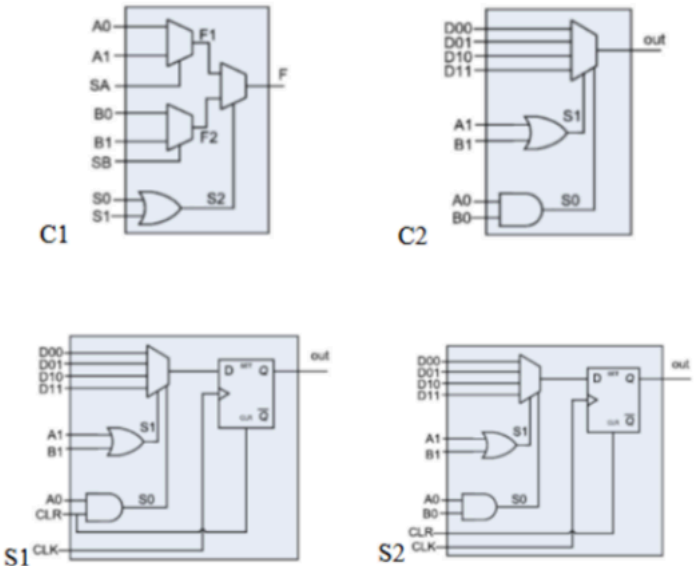
صورت مسئله:

مسئله این تمرین همان مسئله مطرح شده در تمرین اول (طراحی شتاب دهنده شبکه عصبی) است، اما این تمرین برای ساده‌تر شدن، به **دو فاز** شکسته شده است، برای این فاز (فاز ۱) فقط برخی از ماژول‌های تمرین اول را دوباره پیاده سازی می‌کنیم که ماژول‌های مورد نظر در ادامه ذکر شده‌اند و بقیه ماژول‌ها دست نخورده باقی می‌مانند.

آشنایی با ماژول‌های Actel

ماژول های منطقی Actel که در شکل 1 نشان داده شده‌اند ماژول‌های منطقی قابل برنامه‌ریزی هستند. با استفاده از آن‌ها می توان مدار های combinational و sequential مختلفی را پیاده سازی کرد.

هدف این تمرین، پیاده سازی و سنتز طراحی بر روی بخشهای قابل برنامه ریزی یک FPGA است. بنابراین ضروری است تا ماژول های مربوط به این فاز را ابتدا به صورت ساختاری یا همان Structural طراحی کنید و سپس طرح را بر روی سلول های منطقی قابل برنامه ریزی معرفی شده، سنتز کنید. سلولهای منطقی در دسترس در شکل ۱ نمایش داده شده‌اند. دقت کنید که برای طراحی ماژول ها شما تنها مجاز به استفاده از سلول های s2, s1, c2، c1 هستید.



شکل1. سلول های قابل برنامه ریزی مربوط به ماژول Actel

کد مربوط به ماژول های s2, s1, c2، c1 در اختیار شما قرار داده شده است. لازم است بقیه ماژولهای ذکر شده در ادامه را که در پروژه ی یک طراحی کردید با اتصال این ماژول‌ها پیاده‌سازی کنید. لازم به ذکر است که مجاز به استفاده از عمل‌گرهای منطقی در هیچ یک از ماژول‌ها به جز s2, s1، c2، c1 نیستید و فقط مجاز به استفاده از ماژول‌های شکل 1 هستید.

ماژول‌هایی که در این فاز باید پیاده سازی کنید

از بین همه ماژول‌هایی که برای تمرین اول پیاده سازی کردید، ماژول‌های زیر باید با استفاده از ماژول‌های Actel بازطراحی شوند:

- **جمع کننده FP16**

شرایطی که برای موارد خاص باید رعایت شوند:

- در صورت صفر بودن هر یک از مقادیر exponent یا mantissa برای یک ورودی کل ورودی مقدار صفر میگیرد.
- مقادیر subnormal به عنوان صفر لحاظ میشوند.
- تولید carry در جمع مانتیس ها باعث افزایش exponent و شیفت راست مانتیس مقدار نهایی میشود.

- در صورت Normalize نبودن مقدار مانتیس باید Normalize شود و به همان میزان مقدار exponent کاهش پیدا کند.
- در صورت Underflow شدن exponent مقدار نهایی صفر میشود.

- **ضرب کننده FP16**

شرایطی که برای موارد خاص باید رعایت شوند:

- در صورت صفر بودن هر یک از مقادیر exponent یا mantissa برای یک ورودی کل ورودی مقدار صفر میگیرد.
- مقادیر subnormal به عنوان صفر لحاظ میشوند.
- در صورت Normalize نبودن مقدار مانتیس باید Normalize شود و به همان میزان مقدار exponent کاهش پیدا کند.
- در صورت Underflow شدن exponent مقدار نهایی صفر میشود.
- در صورت Overflow شدن exponent مقدار نهایی exponent با مقدار 31 و mantissa با مقدار صفر خواهد داشت.

- **بافر پینگ-پنگی**

پارامترهای ماژول با تمرین یک یکسان میباشد.

همچنین برای راحت‌تر شدن پیاده سازی، در این پروژه نیازی به رند کردن خروجی نیست و به جای آن خروجی همه محاسبات ریاضی را کوتاه (Truncate) کنید.

مواردی که باید حین پیاده سازی به آنها توجه کنید

اکیدا توصیه می شود طراحی به صورت سلسله مراتبی انجام شود. یعنی از ماژول‌های s2 , s1, c2، c1 استفاده و با آنها سایر ماژول ها را به صورت سلسله مراتبی پیاده‌سازی و استفاده کنید.

به صورت آزاد از ماژول ها استفاده نکنید! اگر تعداد گیت‌هایی که برای پیاده سازی هر یک از ماژول ها استفاده کرده‌اید از مقادیر مشخص شده در جدول زیر بیشتر باشد به نسبتی که گیت اضافه استفاده کرده اید از نمره شما کسر خواهد شد.

Module	Limit
Adder	3060
Multiplier	5340
Ping-Pong Buffer	3540

مواردی که باید تحویل دهید:

- طراحی ماژول ها به صورت دستی
- کدهای مربوط به زبان وریلاگ (فقط ماژول‌های مشخص شده برای فاز ۱)
- کدهای تست بنچ (برای هر ماژول یک تست بنچ جداگانه)
- فایل gated.txt به ازای هر ماژول

روش شمردن تعداد ماژول‌ها

کد های **cpp** داده شده را کامپایل کرده و فایل اجرایی (.exe) را در کنار فایل های پروژه بگذارید . لازم به ذکر است اسم فایل های اجرایی باید هم اسم ماژول ها باشد تا دستور $system آن را اجرا کند (اگر اسم فایل خروجی را تغییر دادید باید در دستور سیستم هم تغییر ایجاد کنید). قبل از اجرای simulation فایل gated.txt را نیز در کنار پروژه قرار داده و در آن یک 0 بنویسید تا با اجرا شدن برنامه عدد آن به ازای هر instantiation درون فایل به مقدار تعداد گیت های آن زیاد شود و در پایان simulation تعداد گیت های شما درون فایل نوشته شود (زمان سیمولیشن کمی طولانی تر میشود).

تعداد گیت‌های هر ماژول:

C2 = 11

C1 = 10

S1 = 13

S2 = 13

دستور کامپایل کدهای cpp

```
g++ c1.cpp -o c1.exe

g++ c2.cpp -o c2.exe

g++ s2-s1.cpp -o s2-s1.exe
```

سایر نکات

- انجام این تمرین به صورت گروه‌های دو نفره خواهد بود.
- فایل‌ها و گزارش خود را تا قبل از موعد تحویل، با نام CAD_CA2_<SID1>_<SID2>.zip در محل مربوطه در صفحه درس آپلود کنید.
- ماژولاریتی و تمیزی کدها بخشی از نمره ی پروژه را شامل می‌شود.
- در صورت هرگونه ابهام می‌توانید از طریق ایمیل با طراحان در ارتباط باشید.
- هدف این تمرین یادگیری شماست! در صورت کشف تقلب، مطابق با قوانین درس برخورد خواهد شد.

موفق باشید